

# Python Basic Fresher Interview Questions and Answers

## Preparation guide

**Source and accuracy note:** This is a general Python fresher-level preparation guide based on standard Python concepts and commonly asked entry-level questions. It is not a Wipro internal document, leaked question paper, or verified list of current Wipro questions. Actual questions may vary by role, interviewer, and assessment.

## Python Fundamentals

### 1. What is Python?

Python is a high-level, interpreted, general-purpose programming language known for readable syntax and a large standard library.

### 2. What are important features of Python?

Python is readable, dynamically typed, object-oriented, portable, extensible, and supported by a large ecosystem of libraries.

### 3. Is Python compiled or interpreted?

Python source code is first compiled into bytecode and then executed by the Python virtual machine. It is commonly called interpreted because execution is managed by the interpreter.

### 4. What is a dynamically typed language?

In Python, a variable does not need an explicit type declaration. The type is associated with the object at runtime.

```
value = 10
```

```
value = 'Python'
```

### 5. What is a variable?

A variable is a name that refers to an object. Python variables do not store a fixed declared type.

### 6. What are comments?

A single-line comment starts with #. Triple-quoted strings are commonly used as documentation strings, or docstrings, when placed in the right location.

```
# This is a comment

def greet():
    """This is a docstring."""
    return 'Hello'
```

## 7. What is indentation in Python?

Indentation defines code blocks. Inconsistent indentation can cause an `IndentationError`. Four spaces are the common convention.

## 8. What is PEP 8?

PEP 8 is the main style guide for Python code. It recommends conventions for formatting, naming, imports, and layout.

## 9. What is the difference between syntax errors and exceptions?

A syntax error prevents code from being parsed. An exception occurs during execution, such as attempting to divide by zero.

## 10. What is None?

`None` is Python's singleton object representing the absence of a value. Compare it with `is None` rather than `== None`.

```
result = None

if result is None:
    print('No result')
```

# Data Types and Collections

## 11. Name common built-in data types.

Common types include `int`, `float`, `complex`, `bool`, `str`, `list`, `tuple`, `set`, `dict`, `bytes`, and `NoneType`.

## 12. What is the difference between mutable and immutable objects?

Mutable objects can be changed after creation, such as lists and dictionaries. Immutable objects cannot be changed, such as integers, strings, and tuples.

### **13. What is a list?**

A list is an ordered, mutable collection that can contain duplicate values.

```
numbers = [10, 20, 30]
numbers.append(40)
```

### **14. What is a tuple?**

A tuple is an ordered, immutable collection. It is useful for fixed groups of values.

```
point = (10, 20)
```

### **15. What is a set?**

A set is an unordered collection of unique hashable values.

```
unique_numbers = {1, 2, 2, 3}
print(unique_numbers) # {1, 2, 3}
```

### **16. What is a dictionary?**

A dictionary stores key-value pairs. Keys must be hashable and unique within the dictionary.

```
employee = {'name': 'Ravi', 'salary': 45000}
print(employee['name'])
```

### **17. Difference between list and tuple?**

Lists are mutable and use square brackets. Tuples are immutable and use parentheses. Tuples can be appropriate for fixed data.

### **18. Difference between list and set?**

Lists preserve order and allow duplicates. Sets contain unique values and are optimized for membership checks.

### **19. What is slicing?**

Slicing extracts part of a sequence using start, stop, and optional step. The stop index is excluded.

```
values = [0, 1, 2, 3, 4]
print(values[1:4]) # [1, 2, 3]
print(values[::-1]) # reversed list
```

## 20. What is negative indexing?

Negative indexes count from the end. Index -1 refers to the last element.

## 21. What is a list comprehension?

It is a concise way to create a list from an iterable.

```
squares = [number * number for number in range(1, 6)]
```

## 22. What is a dictionary comprehension?

```
squares = {number: number * number for number in range(1, 4)}
```

## 23. Difference between append and extend?

append adds one object as one element. extend adds each element from an iterable.

```
items = [1, 2]
items.append([3, 4]) # [1, 2, [3, 4]]
items.extend([5, 6]) # [1, 2, [3, 4], 5, 6]
```

## 24. Difference between remove, pop, and del?

remove deletes the first matching value, pop removes and returns an item by index, and del deletes an item, slice, or variable reference.

## 25. What is unpacking?

```
first, second, third = [10, 20, 30]
first, *middle, last = [1, 2, 3, 4]
```

# Operators and Functions

## 26. What is the difference between == and is?

== compares values. is checks whether two references point to the same object.

## 27. What are logical operators?

and, or, and not combine or negate Boolean expressions.

## 28. What are \*args and \*\*kwargs?

\*args collects extra positional arguments into a tuple. \*\*kwargs collects extra keyword arguments into a dictionary.

```
def show_values(*args, **kwargs):
```

```
print(args)
print(kwargs)
```

## 29. What is a function?

A function is a reusable block of code defined with `def`. It can accept parameters and return a value.

```
def add(first, second):
    return first + second
```

## 30. What is a default argument?

```
def greet(name='Guest'):
    return 'Hello, ' + name
```

## 31. What is a lambda function?

A lambda is a small anonymous function containing one expression.

```
double = lambda number: number * 2
```

## 32. What do map, filter, and reduce do?

`map` transforms items, `filter` selects items, and `reduce` combines items into one result. `reduce` is provided by `functools`.

```
from functools import reduce
numbers = [1, 2, 3, 4]
doubled = list(map(lambda number: number * 2, numbers))
even = list(filter(lambda number: number % 2 == 0, numbers))
total = reduce(lambda first, second: first + second, numbers)
```

## 33. What is variable scope?

Scope determines where a name can be accessed. Python commonly follows the LEGB order: Local, Enclosing, Global, Built-in.

## 34. What are global and nonlocal?

`global` allows assignment to a module-level name inside a function. `nonlocal` allows assignment to a name in an enclosing function scope.

## 35. What is recursion?

Recursion is when a function calls itself. It needs a base condition to stop.

```
def factorial(number):  
    if number <= 1:  
        return 1  
    return number * factorial(number - 1)
```

## Object-Oriented Python

### 36. What is a class?

A class is a blueprint for creating objects. It defines data and behavior through attributes and methods.

### 37. What is an object?

An object is an instance of a class with its own state and access to the class behavior.

### 38. What is `__init__`?

`__init__` is an initializer method called after an object is created. It commonly initializes instance attributes.

```
class Employee:  
    def __init__(self, name, salary):  
        self.name = name  
        self.salary = salary
```

### 39. What is `self`?

`self` refers to the current instance inside an instance method. It must be explicitly included in the method definition.

### 40. What is inheritance?

Inheritance allows a child class to reuse or extend behavior from a parent class.

```
class Developer(Employee):  
    pass
```

### 41. What is polymorphism?

Polymorphism allows the same operation or method name to work with objects of different types.

### 42. What is encapsulation?

Encapsulation groups data and methods together and controls how implementation details are accessed. Python uses naming conventions and properties rather than strict private fields.

### **43. What is method overriding?**

Method overriding occurs when a child class defines a method with the same name as a parent method.

### **44. Difference between instance, class, and static methods?**

An instance method receives self. A class method receives cls and is marked with @classmethod. A static method receives neither automatically and is marked with @staticmethod.

### **45. What is multiple inheritance?**

Multiple inheritance means a class inherits from more than one parent class. Python resolves method lookup using its method resolution order.

## **Exceptions, Files, and Modules**

### **46. What is exception handling?**

Exception handling manages runtime errors using try, except, else, and finally.

```
try:
    result = 10 / 0
except ZeroDivisionError:
    result = 0
finally:
    print('Finished')
```

### **47. Difference between except and finally?**

except handles a matching exception. finally runs whether an exception occurs or not and is commonly used for cleanup.

### **48. What does raise do?**

raise explicitly signals an exception.

```
if age < 0:
    raise ValueError('Age cannot be negative')
```

## 49. How do you open a file safely?

Use a with statement so the file is closed automatically.

```
with open('data.txt', 'r', encoding='utf-8') as file:
    content = file.read()
```

## 50. What is a module?

A module is a Python file containing definitions and statements that can be imported into another file.

```
import math
print(math.sqrt(25))
```

## 51. What is a package?

A package is a directory used to organize related modules. Modern Python packages may use an `__init__.py` file, although namespace packages can work without one.

## 52. What is the purpose of `if __name__ == '__main__':`?

It runs code only when the file is executed directly, not when it is imported.

```
if __name__ == '__main__':
    print('Program started directly')
```

## 53. What is pip?

pip is a package installer for Python packages, usually from the Python Package Index.

## 54. What is a virtual environment?

A virtual environment isolates project dependencies from other Python projects.

```
python -m venv .venv
```

# Common Coding Questions

## 55. Reverse a string.

```
text = 'python'
reversed_text = text[::-1]
```

## 56. Check whether a string is a palindrome.

```
text = 'madam'
is_palindrome = text == text[::-1]
```

### **57. Count vowels in a string.**

```
text = 'education'
vowel_count = sum(1 for character in text.lower() if character in 'aeiou')
```

### **58. Find the largest number in a list.**

```
numbers = [12, 5, 20, 8]
largest = max(numbers)
```

### **59. Find duplicate values in a list.**

```
numbers = [1, 2, 2, 3, 4, 4]
duplicates = {number for number in numbers if numbers.count(number) > 1}
```

For large lists, a frequency dictionary or Counter is more efficient than repeatedly calling count.

### **60. Count the frequency of list values.**

```
from collections import Counter
numbers = [1, 2, 2, 3, 3, 3]
frequency = Counter(numbers)
```

### **61. Check whether a number is prime.**

```
def is_prime(number):
    if number < 2:
        return False
    for divisor in range(2, int(number ** 0.5) + 1):
        if number % divisor == 0:
            return False
    return True
```

### **62. Print Fibonacci numbers.**

```
def fibonacci(count):
    first, second = 0, 1
    result = []
    for _ in range(count):
        result.append(first)
        first, second = second, first + second
```

```
    return result
```

### **63. Swap two variables.**

```
first, second = second, first
```

### **64. Sort a list.**

```
numbers = [3, 1, 2]
```

```
sorted_numbers = sorted(numbers) # creates a new list
```

```
numbers.sort() # changes the existing list
```

### **65. What should a fresher practice?**

Practice variables, strings, lists, tuples, sets, dictionaries, loops, functions, exceptions, file handling, OOP, and simple coding problems. Be ready to explain time complexity and edge cases for each solution.

## **Final Preparation Checklist**

Revise Python syntax, indentation, data types, mutability, collections, functions, OOP, exceptions, modules, file handling, and basic problem solving. Practice writing code without relying only on memorized solutions.